

Rilevamento e mitigazione DDoS in SDN tramite Machine Learning

Network and Cloud Infrastructures - Prof. Giorgio Ventre - a.a. 2025/2026
Francesco Beneduce & Emerico Buonanno

1. Obiettivo del progetto e approccio seguito

Il progetto realizza una pipeline SDN completa per difendere un server da traffico DDoS UDP. La rete viene emulata in Mininet, il controllo è affidato a Ryu e le decisioni di sicurezza sono prese a partire dalle statistiche OpenFlow raccolte periodicamente dagli switch. Il controller non blocca intere porte o interi switch: quando una sorgente è ritenuta malevola installa una regola DROP selettiva solo sulla coppia sorgente-vittima, lasciando il resto della rete operativo.

La scelta del Machine Learning nasce da un vincolo progettuale preciso: nel dataset finale il traffico legittimo e quello d'attacco hanno rate volutamente sovrapposti. In questo modo il problema non è risolvibile con una semplice soglia sul packet-rate. Il modello deve combinare informazioni del singolo flusso, andamento temporale e contesto aggregato verso la vittima.

2. Topologia core/access

La topologia adotta uno switch centrale s0 e quattro switch di accesso s1-s4. Tutti i link hanno banda 10 Mbit/s e delay 5 ms. La vittima è h4 (10.0.0.4), collegata a s2. Nel caso distribuito gli attaccanti h3, h5, h9 e h12 si trovano su switch differenti, così il traffico malevolo arriva da più punti della rete e attraversa il core.

Topologia core/access SDN

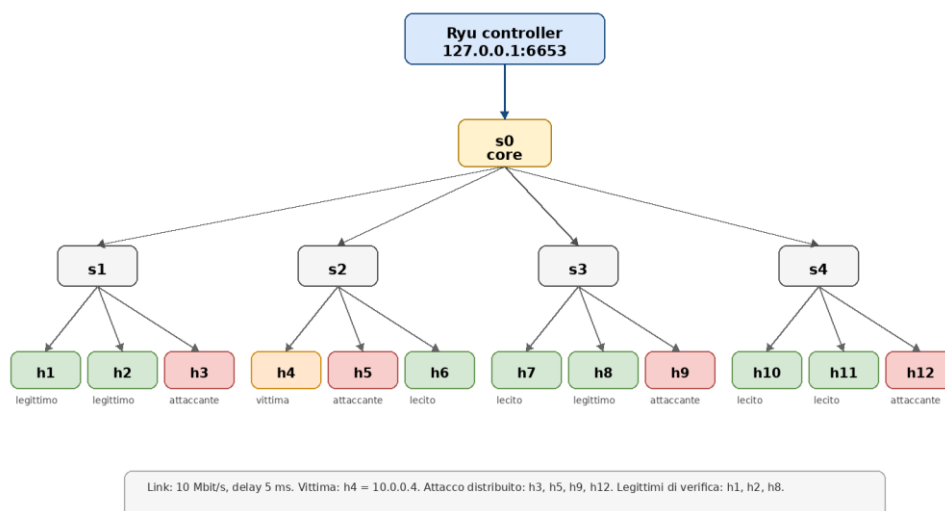


Figura 1 - Topologia SDN core/access: h4 è la vittima; h3, h5, h9 e h12 sono le sorgenti malevole nello scenario distribuito.

3. Generazione del traffico e labeling

Il traffico è generato con `udp_realistic_sender.py`, uno script Python basato su socket UDP. Lo stesso programma viene usato sia per host legittimi sia per attaccanti: cambiano i parametri di rate, payload, burst e jitter. Questo evita un flood artificiale e rende l'esperimento più realistico, perché il volume del singolo flusso non basta da solo a separare le classi.

Sulla vittima h4 viene avviato `udp_sink.py`, che apre le porte UDP e scarta i pacchetti ricevuti. Il sink evita che il kernel risponda con ICMP port unreachable quando arrivano datagrammi verso porte chiuse; in questo modo il dataset rimane pulito e costituito da traffico UDP.

4. Dataset raccolto

Il dataset è stato raccolto dagli autori eseguendo 15 sessioni sperimentali: traffico normale, attacco singolo e attacco distribuito, ripetuti su cinque round. Dopo la pulizia vengono usate solo le righe attive, cioè quelle con incremento reale di pacchetti nel ciclo di polling.

Dato	Valore
Righe CSV totali	4703
Label 0 - traffico normale	3052
Label 1 - attacco	1651
Righe attive usate nel training	3942 ($\text{delta_packets} > 0$)
Sessioni sperimentali	15: normale, attacco singolo, attacco distribuito su 5 round
Protocollo del dataset	UDP, proto=17

5. Feature: flusso singolo e contesto aggregato

Il modello usa 16 feature, coerenti tra `train_model.py`, `best_model.joblib`, `model_metadata.json` e controller a runtime. Sono escluse porte L4 e contatori cumulativi grezzi, perché avrebbero reso il modello meno generalizzabile e più dipendente dallo specifico esperimento.

Feature del singolo flusso <code>delta_packets</code> , <code>delta_bytes</code> , <code>packet_rate</code> , <code>byte_rate</code> , <code>burst_score</code> .	Feature aggregate verso h4 <code>dst_unique_src_count</code> , <code>dst_total_packet_rate</code> , <code>dst_total_byte_rate</code> , <code>dst_active_flow_count</code> , <code>dst_std_packet_rate</code> , <code>dst_cv_packet_rate</code> , <code>flow_to_dst_ratio</code> , <code>pps_per_source</code> .
---	---

Il punto più importante è l'unione tra feature locali e feature aggregate. Le prime descrivono il comportamento del singolo flusso; le seconde descrivono cosa sta accadendo alla vittima nel suo insieme. In un DDoS distribuito questa informazione è decisiva, perché ogni sorgente può sembrare moderata se vista da sola, ma l'insieme delle sorgenti crea saturazione verso h4.

6. Addestramento del modello e scelta della soglia

Il classificatore finale è un Random Forest. Gli iperparametri sono selezionati tramite grid search; la configurazione finale usa `max_depth=8`, `n_estimators=150` e `min_samples_leaf=2`. La soglia operativa scelta è 0.40: non massimizza soltanto l'accuracy, ma mantiene recall molto alto sugli attacchi, scelta coerente con un sistema di mitigazione DDoS.

Accuracy	Precision	Recall	F1
0.972	0.938	0.988	0.963

7. Risultati del training e confronto con la baseline

	Predetto normale	Predetto attacco
Reale normale	717	28
Reale attacco	5	426

La matrice di confusione mostra 426 attacchi rilevati su 431 e 28 falsi positivi su 745 normali. Le metriche non sono più artificialmente perfette: sono realistiche e più difendibili, perché il dataset contiene traffico legittimo sovrapposto agli attacchi e non permette scorciatoie banali.

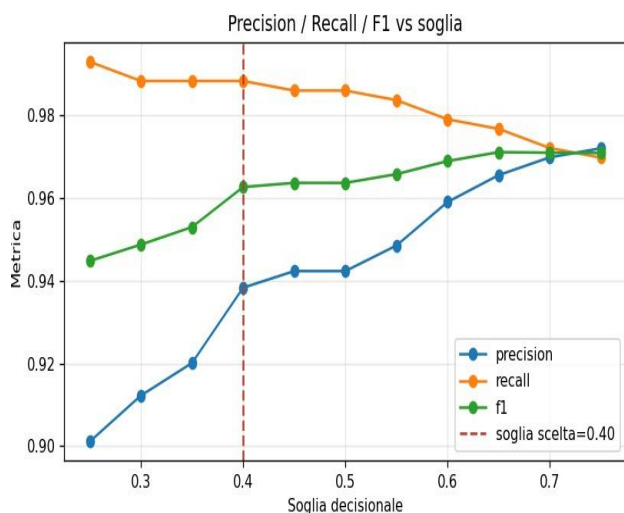


Figura 2 - Andamento precision, recall e F1 al variare della soglia.

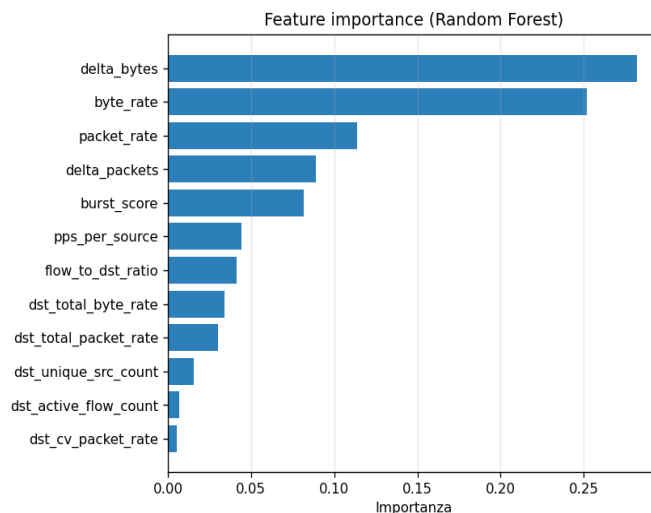


Figura 3 - Feature importance: prevalgono byte/rate, delta e burst_score.

I grafici confermano che il modello non sta classificando in base a IP o porte hard-coded: le feature più pesanti descrivono variazioni di byte e pacchetti, intensità, burst e rapporto tra il singolo flusso e il traffico complessivo diretto alla vittima.

8. Detection runtime e mitigazione OpenFlow

A runtime il controller invia richieste `OFFlowStatsRequest` agli switch, calcola feature incrementali e aggrega lo stato della vittima. La decisione finale non dipende solo dalla probabilità prodotta dal modello: per installare una regola DROP servono anche un contesto aggregato compatibile con un attacco e quattro conferme consecutive sullo stesso flusso.

Pipeline detection e mitigazione

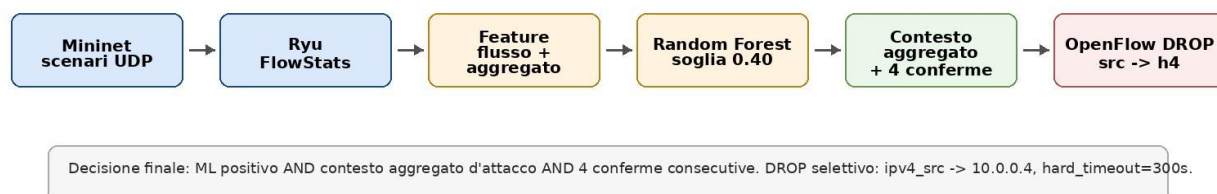


Figura 4 - Pipeline runtime: Mininet genera gli scenari, Ryu raccoglie FlowStats, il modello classifica e OpenFlow applica DROP selettivi.

Logica di blocco: predizione ML positiva AND contesto aggregato di attacco verso 10.0.0.4 AND SUSPICIOUS_LIMIT=4. La regola installata ha priority=200, hard_timeout=300s e match su ipv4_src -> 10.0.0.4. Il ritardo iniziale è voluto: riduce i blocchi impulsivi su traffico legittimo bursty.

9. Verifica sperimentale in Mininet

I test verificano tre scenari: traffico normale, attacco singolo e attacco distribuito. Durante gli attacchi il traffico legittimo resta attivo, perché l'obiettivo non è isolare la vittima ma neutralizzare le sorgenti malevole continuando a servire gli utenti leciti. Nel distribuito il sistema blocca stabilmente la maggioranza degli attaccanti; questo basta a ridurre il volume aggregato sotto la soglia critica.

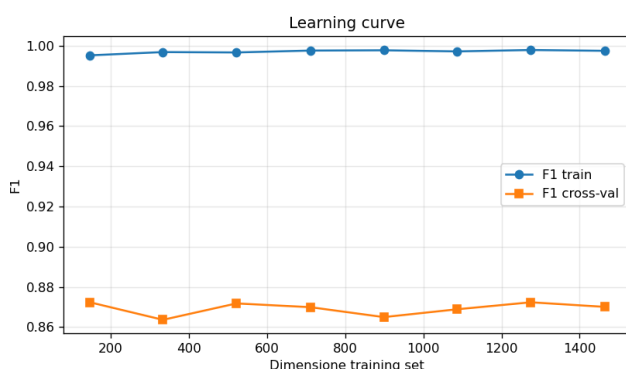


Figura 5 - Learning curve: il modello mantiene prestazioni stabili al crescere dei dati.

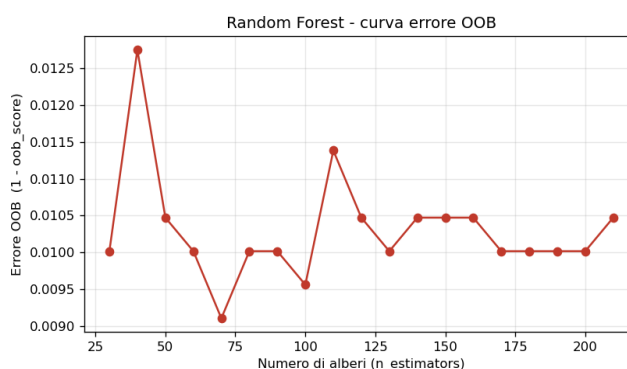


Figura 6 - Errore OOB della Random Forest al variare degli alberi.

L'esito atteso della demo è : h3, h5, h9 e h12 subiscono perdita elevata o 100% verso h4 dopo l'installazione dei DROP; h1, h2 e h8 mantengono 0% packet loss. Nei dump OpenFlow non devono comparire regole DROP con nw_src=10.0.0.1, 10.0.0.2 o 10.0.0.8.

10. Comandi di esecuzione

La demo viene eseguita con due terminali. Il primo mantiene il controller Ryu aperto e permette di osservare i log “Flow”, “ml=”, “agg=” e “final=”, mentre il secondo avvia la topologia Mininet e i test automatici.

```

Terminale 1 - controller
cd ~/Scrivania/DDOS_progetto
sudo mn -c
PYTHONPATH=. /ryu python3 ryu/bin/ryu-manager
controller/ml_detector_controller_final.py
Terminale 2 - topologia Mininet
cd ~/Scrivania/DDOS_progetto
cp scripts/udp_realistic_sender.py scripts/udp_sink.py
/tmp/ sudo -E python3 topology/network_core_access.py
Dentro mininet>
source scripts/test_in_mininet/verify_full.mn

```

11. Limiti e discussione critica

Il sistema è validato su flood UDP, singolo e distribuito. TCP, ICMP e attacchi applicativi restano sviluppi futuri. La rete è emulata in Mininet, quindi l’esperimento è riproducibile ma non sostituisce una validazione su rete reale. Inoltre, nei primi secondi dell’attacco, il controller attende quattro conferme prima di bloccare: questo introduce un breve transitorio, ma protegge i legittimi da decisioni troppo aggressive.

12. Conclusioni

Il progetto costruisce una pipeline SDN completa: Mininet genera gli scenari, Ryu raccoglie i flussi, Random Forest classifica il traffico e OpenFlow applica mitigazioni selettive verso h4. Rendendo l’attacco più realistico - rate sovrapposto, sorgenti distribuite e traffico UDP pulito - il problema diventa non banale e giustifica l’uso del ML, come dimostrato dal confronto con la baseline. Il controller combina feature del singolo flusso, contesto aggregato e conferme consecutive: così blocca la maggioranza delle sorgenti d’attacco senza colpire stabilmente i legittimi. I limiti principali sono lo scope UDP e il transitorio iniziale prima della mitigazione.

13. File principali del repository

```

controller/ml_detector_controller_final.py - detection e
regole DROP
controller/dataset_collector_auto.py      - raccolta dataset
topology/network_core_access.py          - topologia core/access
scripts/udp_realistic_sender.py          - generatore UDP
scripts/udp_sink.py                      - sink UDP della vittima
scripts/test_in_mininet/verify_full.mn    - demo completa
dataset/dataset_v8_training.csv           - dataset raccolto
ml/best_model.joblib                     - modello finale
results/*.png                             - grafici e risultati training

```